PharmaChat

Documentation Technique Complète

Interface de gestion des demandes pharmaceutiques via WhatsApp & Mobile

Next.js 16 | TypeScript | Prisma ORM | MongoDB | Socket.io | Twilio WhatsApp | Tailwind CSS v4

Auteur : Oussama Salmi | **Date :** Avril 2026

Projet réalisé dans le cadre du stage de fin d'études

Table des matières

1. Vue d'ensemble du projet
   - — Problématique & objectif
   - — Architecture globale
2. Stack technique
   - — Pourquoi chaque outil ?
3. Structure des fichiers
4. Base de données — Prisma & MongoDB
   - — Modèles Patient & Message
   - — Connexion Prisma
5. Couche données — lib/data.ts
   - — Types TypeScript
   - — Fonctions publiques
6. API REST — Routes Next.js
   - — GET & POST /api/patients/[id]/messages
   - — POST /api/webhook (Twilio)
7. Temps réel — Socket.io
   - — Serveur custom (server.js)
   - — Client React
8. Intégration WhatsApp — Twilio
   - — Envoi de messages
   - — Réception via webhook
9. Composants React
   - — Server Components vs Client Components
   - — Détail de chaque composant
10. Flux complet de données
11. Concepts clés expliqués
12. Comment lancer le projet

1 Vue d'ensemble du projet

Problématique

Les pharmaciens reçoivent chaque jour des demandes de médicaments via WhatsApp et des applications mobiles. Gérer ces conversations en passant d'un écran à l'autre est fastidieux et source d'erreurs.

Solution : PharmaChat

PharmaChat est une interface web de style **WhatsApp Web** dédiée aux pharmaciens. Elle centralise toutes les conversations patients dans une seule application, avec :

- **Sidebar** — liste de tous les patients avec indicateurs de non-lus
- **Zone de chat** — historique complet des messages, envoi en temps réel
- **Actions rapides** — réponses prédéfinies (Disponible, Rupture, Ordonnance...)
- **Temps réel** — les nouveaux messages WhatsApp apparaissent immédiatement sans rechargement

Architecture globale

```
┌──────────────────────────────────────────────────────────────────────┐
│                          PHARMACHAT                                  │
│                                                                      │
│  ┌──────────────┐   HTTP/WebSocket   ┌──────────────────────────┐   │
│  │   Navigateur │◄──────────────────►│  Next.js + Socket.io     │   │
│  │   (React UI) │                    │  (server.js — port 3000) │   │
│  └──────────────┘                    └──────────┬───────────────┘   │
│                                                  │                   │
│                                              Prisma ORM              │
│                                                  │                   │
│                                     ┌────────────▼──────────────┐    │
│                                     │       MongoDB Atlas       │    │
│                                     │  (patients + messages)    │    │
│                                     └───────────────────────────┘    │
│                                                                      │
│  ┌──────────────┐   HTTPS POST       ┌──────────────────────────┐   │
│  │    Twilio    │──────────────────►│  /api/webhook            │   │
│  │  (WhatsApp)  │                    │  (reçoit les messages)   │   │
│  └──────────────┘                    └──────────────────────────┘   │
│                                                                      │
│  ┌──────────────┐   API Twilio       ┌──────────────────────────┐   │
│  │    Patient   │◄──────────────────│  /api/patients/[id]/msgs  │   │
│  │   WhatsApp   │                    │  (envoi de messages)     │   │
│  └──────────────┘                    └──────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

2 Stack technique

Technologie	Version	Rôle	Pourquoi ce choix ?
Next.js	16.2.3	Framework React full-stack	App Router intégré, routes API sans backend séparé, rendu côté serveur
TypeScript	5.x	Typage statique	Détecte les erreurs à la compilation, autocomplétion IDE, maintenabilité
Tailwind CSS	4.x	Styles utilitaires	Pas de fichiers CSS séparés, cohérence visuelle, responsive facile
Prisma ORM	6.x	Accès base de données	Schéma type-safe, migrations auto, supporte MongoDB
MongoDB	Atlas	Base de données	Stockage flexible (tableaux de médicaments), hébergement cloud gratuit
Socket.io	4.x	Communication temps réel	WebSocket avec fallback polling, notifications push instantanées
Twilio	API WhatsApp	Intégration WhatsApp	API officielle WhatsApp Business, fiable et bien documentée

3 Structure des fichiers

```
pharmachat/
├── app/ — Pages Next.js (App Router)
│ ├── layout.tsx — Layout racine (Sidebar + contenu)
│ ├── page.tsx — Page d'accueil (statistiques)
│ ├── chat/[patientId]/
│ │ └── page.tsx — Page de conversation dynamique
│ └── api/
│ ├── patients/
│ │ ├── route.ts — GET tous les patients
│ │ └── [id]/messages/
```

```
│ │ └── route.ts — GET messages / POST envoi
│ └── webhook/
│ └── route.ts — Réception messages WhatsApp (Twilio)
├── components/ — Composants React réutilisables
│ ├── Sidebar.tsx — Server Component (chargement données)
│ ├── SidebarClient.tsx — Client Component (recherche, Socket.io)
│ ├── ChatContainer.tsx — Client Component (messages, envoi)
│ ├── ChatHeader.tsx — En-tête patient
│ └── QuickActions.tsx — Boutons de réponses rapides
├── lib/ — Logique partagée
│ ├── data.ts — Types + requêtes Prisma → UI
│ ├── prisma.ts — Singleton Prisma Client
│ ├── socket-server.ts — Accès à l'instance Socket.io serveur
│ └── socket-client.ts — Singleton Socket.io côté navigateur
├── prisma/
│ └── schema.prisma — Schéma base de données
├── server.js — Serveur HTTP custom (Next.js + Socket.io)
└── .env — Variables d'environnement (secrets)
```

4 Base de données — Prisma & MongoDB

Qu'est-ce que Prisma ?

Prisma est un **ORM** (Object-Relational Mapper). Il transforme le schéma déclaratif `schema.prisma` en un client TypeScript typé qui permet d'interroger MongoDB avec des fonctions simples au lieu d'écrire du SQL ou des requêtes Mongo brutes.

Schéma — `prisma/schema.prisma`

```
// Connexion à MongoDB
datasource db {
  provider = "mongodb"
  url      = env("DATABASE_URL")
}

// Modèle Patient
model Patient {
  id          String    @id @default(auto()) @map("_id") @db.ObjectId
  phone       String    @unique          // Numéro WhatsApp unique
  name        String
  channel     String    @default("whatsapp")
  status      String    @default("offline")
  lastSeen    String    @default("Hors ligne")
  unreadCount Int       @default(0)
  medications String[]                  // Tableau — avantage MongoDB
  avatarColor String    @default("#667781")
  messages    Message[]                 // Relation 1→N
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

// Modèle Message
model Message {
  id          String   @id @default(auto()) @map("_id") @db.ObjectId
  content     String
  channel     String   @default("whatsapp")
  isSender    Boolean  @default(false)   // true = pharmacien, false = patient
  status      String   @default("delivered")
  whatsappId  String?                   // MessageSid Twilio (déduplication)
  patientId   String   @db.ObjectId
  patient     Patient  @relation(fields: [patientId], references: [id])
  createdAt   DateTime @default(now())
}
```

Relation Patient → Message

Un patient possède plusieurs messages (**relation 1 → N**). Chaque message contient `patientId` qui pointe vers le patient. Prisma gère cette jointure automatiquement via `include: { messages: true }`.

Connexion Prisma — `lib/prisma.ts`

En développement, Next.js recharge les modules fréquemment. Sans précaution, chaque rechargement créerait une nouvelle connexion MongoDB. Le pattern **singleton global** résout ce problème :

```
import { PrismaClient } from '@prisma/client'

// Stocké dans globalThis pour survivre aux hot-reloads
const globalForPrisma = global as { prisma?: PrismaClient }

export const prisma = globalForPrisma.prisma ?? new PrismaClient()

if (process.env.NODE_ENV !== 'production')
  globalForPrisma.prisma = prisma
```

5 Couche données — `lib/data.ts`

Types TypeScript

Ce fichier définit les types que *tous* les composants utilisent :

```
export type Channel       = 'whatsapp' | 'mobile'
export type MessageSender = 'patient' | 'pharmacist'
export type MessageStatus = 'sent' | 'delivered' | 'read'

export interface Message {
  id:        string
  content:   string
  sender:    MessageSender
  timestamp: string   // ISO 8601 — ex: "2026-04-20T10:30:00Z"
  status:    MessageStatus
}

export interface Patient {
  id:              string
  name:            string
  phone:           string
  status:          'online' | 'offline'
  lastSeen:        string
  unreadCount:     number
  medications:     string[]
  channel:         Channel
  lastMessage:     string
  lastMessageTime: string
  messages:        Message[]
  avatarColor:     string
}
```

Fonctions publiques

Fonction	Ce qu'elle fait	Utilisée dans
`getPatients()`	Récupère tous les patients triés par date de dernier message, avec leurs messages	`Sidebar.tsx`
`getPatientById(id)`	Récupère un patient précis avec tous ses messages pour afficher la conversation	`app/chat/[patientId]/page.tsx`

Transformation Prisma → UI

Prisma stocke `isSender: boolean` et `createdAt: Date`. L'UI attend `sender: 'patient'|'pharmacist'` et `timestamp: string`. La fonction interne `toUIMessage()` fait cette conversion de manière transparente.

6 API REST — Routes Next.js

Next.js App Router permet de définir des routes API dans des fichiers `route.ts`. Chaque export (`GET`, `POST`...) correspond au verbe HTTP.

GET /api/patients/[id]/messages

Retourne l'historique des messages d'un patient depuis MongoDB.

```
export async function GET(_req, { params }) {
  const { id } = await params

  const rows = await prisma.message.findMany({
    where:   { patientId: id },
    orderBy: { createdAt: 'asc' },
  })

  return NextResponse.json(rows.map(toUIMessage))
}
```

POST /api/patients/[id]/messages

Envoie un message au patient. Fait 3 choses en séquence :

1. Valide le contenu (non vide) et vérifie que le patient existe en base
2. Sauvegarde le message dans MongoDB (`isSender: true`)
3. Envoie le message WhatsApp via l'API Twilio si les variables d'env sont configurées

```
// Étape 2 : sauvegarde en base
const row = await prisma.message.create({
  data: {
    content,
    channel:  patient.channel,
    isSender: true,    // pharmacien envoie
    status:   'sent',
    patientId: id,
  },
})

// Étape 3 : envoi WhatsApp via Twilio
const client = twilio(accountSid, authToken)
await client.messages.create({
  from: `whatsapp:${from}`,
  to:   `whatsapp:${patient.phone}`,
  body: content,
})
```

POST /api/webhook — Réception WhatsApp

Twilio appelle cette URL automatiquement dès qu'un patient envoie un WhatsApp. C'est le point d'entrée de tous les messages entrants.

1. **Parse** le corps URL-encodé envoyé par Twilio (`From` , `Body` , `MessageSid`)
2. **Déduplication** : si le `MessageSid` existe déjà, on ignore (Twilio peut retenter)
3. **Upsert patient** : crée le patient s'il est inconnu, sinon met à jour son statut
4. **Sauvegarde** le message en base (`isSender: false`)
5. **Incrémente** le compteur de messages non lus
6. **Émet** via Socket.io pour mise à jour instantanée de l'UI

```
// Étape 6 : notification temps réel
const io = getIO()
if (io) {
  io.emit('new_message',   { patientId: patient.id, message: messageForUI })
  io.emit('unread_update', { patientId: patient.id, unreadCount: updated.unreadCount })
}
```

7 Temps réel — Socket.io

Pourquoi Socket.io ?

HTTP est unidirectionnel : le client demande, le serveur répond. Pour recevoir des messages sans recharger la page, il faut une connexion **bidirectionnelle persistante** — c'est ce que WebSocket (et Socket.io) fournit.

> **Contrainte Next.js**
>
> Next.js ne supporte pas nativement Socket.io dans son serveur interne. La solution : un **serveur HTTP custom** (`server.js`) qui encapsule Next.js et Socket.io sur le même port.

Serveur — `server.js`

```
const { createServer } = require('http')
const { Server }       = require('socket.io')
const next             = require('next')

const app        = next({ dev })
const httpServer = createServer(handle)   // Next.js gère les requêtes HTTP

const io = new Server(httpServer, {
  cors: { origin: '*' },
})

// Accessible depuis les routes API via globalThis.io
globalThis.io = io

io.on('connection', (socket) => {
  console.log('[Socket.io] Client connecté:', socket.id)
})

httpServer.listen(3000)
```

Accès depuis les routes API — `lib/socket-server.ts`

```
export function getIO(): Server | null {
  return globalThis.io ?? null
}
```

Client React — `lib/socket-client.ts`

Singleton pour éviter de créer plusieurs connexions WebSocket lors des re-renders React :

```
let socket: Socket | null = null

export function getSocket(): Socket {
  if (!socket) {
    socket = io({ path: '/socket.io', transports: ['websocket', 'polling'] })
  }
  return socket
}
```

Événements Socket.io

Événement	Émis par	Reçu par	Données
`new_message`	Webhook API	ChatContainer, SidebarClient	`{ patientId, message }`
`unread_update`	Webhook API	SidebarClient	`{ patientId, unreadCount }`

8 Intégration WhatsApp — Twilio

Comment fonctionne Twilio ?

Twilio est un intermédiaire entre WhatsApp et notre application. Le pharmacien n'a pas besoin d'un compte WhatsApp Business directement — Twilio fournit un numéro sandbox pour les tests.

```
PATIENT                    TWILIO                    PHARMACHAT
  |                          |                          |
  |── "Bonjour, j'ai besoin ──►|                          |
  |      de Doliprane"         |                          |
  |                          |── POST /api/webhook ──────►|
  |                          |   (Body, From, MessageSid) |
  |                          |                          |── Sauvegarde MongoDB
  |                          |                          |── Émet Socket.io
  |                          |                          |── ✅ 200 OK ──────────►|
  |                          |                          |
  |                          |                          |  (pharmacien répond)
  |                          |◄── API Twilio (POST) ──────|
  |◄── "Disponible ! ─────────|                          |
  |     En stock"              |                          |
```

Variables d'environnement requises

```
# .env
DATABASE_URL="mongodb+srv://user:pass@cluster.mongodb.net/pharmachat"

TWILIO_ACCOUNT_SID="ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
TWILIO_AUTH_TOKEN="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
TWILIO_WHATSAPP_NUMBER="+14155238886"  # Numéro sandbox Twilio
```

Configuration du webhook Twilio

Dans la console Twilio, renseigner l'URL du webhook : `https://votre-domaine.com/api/webhook`

En développement local

Twilio ne peut pas atteindre `localhost`. Utiliser **ngrok** pour exposer le port 3000 publiquement :
`ngrok http 3000` → copier l'URL `https://xxxx.ngrok.io` dans la console Twilio.

9 Composants React

Server Components vs Client Components

Next.js App Router introduit deux types de composants. La règle est simple : **tout est Server Component par défaut**. On ajoute `'use client'` uniquement si nécessaire.

Type	Directive	Capacités	Exemple dans PharmaChat
Server Component	aucune	Accès DB, async/await, pas d'état React	`Sidebar.tsx`, `ChatHeader.tsx`
Client Component	`'use client'`	useState, useEffect, événements, Socket.io	`SidebarClient.tsx`, `ChatContainer.tsx`

```
Sidebar.tsx (Server)          SidebarClient.tsx (Client)
____________________          ______________________________
await getPatients()    ──►    useState(initialPatients)
(accès MongoDB)               useEffect → Socket.io listen
                              search filter
                              Link navigation
```

Détail de chaque composant

Sidebar.tsx — Server Component

Charge la liste des patients depuis MongoDB et transmet à `SidebarClient`.

```
export default async function Sidebar() {
  const patients = await getPatients()  // requête MongoDB côté serveur
  return <SidebarClient patients={patients} />
}
```

SidebarClient.tsx — Client Component

- **Recherche** : filtre par nom de patient ou médicament (useState)
- **Navigation active** : surligne la conversation ouverte (usePathname)

- **Temps réel** : écoute `new_message` et `unread_update` via Socket.io
- **Badges** : icône WhatsApp/Mobile, compteur non-lus, aperçu dernier message

ChatContainer.tsx — Client Component

Le composant principal de conversation. Gère :

- **Affichage des messages** avec séparateurs de dates ("Aujourd'hui", "Hier"...)
- **Bulles de chat** : vert clair pour le pharmacien, gris pour le patient
- **Statut des messages** : ✓ (envoyé), ✓✓ gris (reçu), ✓✓ bleu (lu)
- **Auto-scroll** vers le dernier message
- **Envoi optimiste** : le message apparaît immédiatement avant la réponse serveur
- **Socket.io** : reçoit les nouveaux messages en temps réel

QuickActions.tsx — Client Component

5 boutons de réponses prédéfinies cliquables par le pharmacien :

Bouton	Message envoyé
Disponible	Ce médicament est disponible en pharmacie.
Rupture de stock	Ce médicament est en rupture de stock. Nous vous contacterons dès sa disponibilité.
Ordonnance requise	Ce médicament nécessite une ordonnance médicale valide.
Commander	Nous allons commander ce médicament pour vous. Délai estimé : 24-48h.
Informations	Pour plus d'informations, merci de vous présenter à la pharmacie.

ChatHeader.tsx — Server Component

Affiche le nom du patient, son numéro, son statut en ligne, ses médicaments, et le canal de communication.

10 Flux complet de données

Scénario A : Un patient envoie un message WhatsApp

1. **Patient → WhatsApp → Twilio**
   Le patient écrit "Bonjour, avez-vous du Doliprane ?" sur WhatsApp. Twilio reçoit ce message.

2. **Twilio → POST /api/webhook**
   Twilio envoie une requête HTTP POST à notre webhook avec les paramètres :
   `From=whatsapp:+212601234567`, `Body=Bonjour, avez-vous du Doliprane ?`, `MessageSid=SM...`

3. **Webhook → MongoDB**
   Le webhook crée ou met à jour le patient, sauvegarde le message, incrémente `unreadCount`.

4. **Webhook → Socket.io → React**
   Le webhook émet `new_message` et `unread_update`. `ChatContainer` et `SidebarClient` reçoivent ces événements et mettent à jour leur état React instantanément.

5. **Affichage immédiat**
   Sans rechargement : le message apparaît dans la conversation, et le badge rouge s'incrémente dans la sidebar.

Scénario B : Le pharmacien envoie une réponse

1. **Saisie → Mise à jour optimiste**
   Le pharmacien tape et appuie sur Entrée. `ChatContainer` ajoute immédiatement le message dans l'UI avec un ID temporaire (`temp-1234`), sans attendre le serveur.

2. **POST /api/patients/[id]/messages**
   En parallèle, une requête API est envoyée. Le serveur sauvegarde en MongoDB et appelle l'API Twilio.

3. **Réponse → Remplacement**
   À réception de la réponse 201, le message temporaire est remplacé par le message sauvegardé (avec le vrai ID MongoDB).

4. **Twilio → Patient WhatsApp**
   Twilio délivre le message sur le téléphone du patient via WhatsApp.

11 Concepts clés expliqués

App Router de Next.js

L'App Router remplace l'ancien Pages Router. Chaque dossier dans `app/` devient une route URL. Le fichier `page.tsx` est le composant rendu. `[patientId]` entre crochets est un paramètre dynamique.

Dossier	URL	Rendu
`app/page.tsx`	`/`	Page d'accueil
`app/chat/[patientId]/page.tsx`	`/chat/6819abc...`	Conversation d'un patient
`app/api/patients/route.ts`	`/api/patients`	API JSON

Mise à jour optimiste (Optimistic Update)

Technique UX qui affiche le résultat d'une action *avant* que le serveur confirme. Si le serveur échoue, on peut rollback. Ici, le message apparaît instantanément, puis est remplacé par la version confirmée.

> **Résultat perçu**
>
> L'expérience est identique à WhatsApp : le message s'affiche instantanément dès la frappe, sans latence réseau visible.

WebSocket vs HTTP polling

	HTTP Polling	WebSocket (Socket.io)
Principe	Le client demande toutes les X secondes	Connexion persistante bidirectionnelle
Latence	Jusqu'à X secondes	Quasi-instantanée (<50ms)
Charge serveur	Élevée (N clients × N requêtes/min)	Faible (1 connexion par client)
Utilisé dans PharmaChat	Fallback automatique	Mode principal

Singleton Pattern

Utilisé deux fois dans ce projet :

- **PrismaClient** — une seule instance de connexion DB pour toute l'application
- **Socket.io Client** — une seule connexion WebSocket par onglet navigateur

Sans ce pattern, chaque re-render React ou rechargement Next.js créerait de nouvelles connexions, épuisant les ressources.

ORM (Object-Relational Mapper)

Prisma traduit des appels TypeScript en requêtes MongoDB. Avantages :

- Autocomplétion complète dans l'IDE (`prisma.patient.findMany({...})`)
- Types inférés automatiquement — pas d'erreurs de champ inexistant
- Migrations de schéma gérées automatiquement (`prisma db push`)

Déduplication de messages

Twilio peut retenter l'envoi du webhook si la réponse tarde. Sans déduplication, le même message serait sauvegardé deux fois. La vérification sur `whatsappId` (MessageSid unique) évite ce doublon :

```
const existing = await prisma.message.findFirst({ where: { whatsappId: messageSid } })
if (existing) return new NextResponse('', { status: 200 })  // Ignorer le doublon
```

12 Comment lancer le projet

Prérequis

- Node.js 18+ installé
- Un compte MongoDB Atlas (gratuit) avec une base `pharmachat`
- Un compte Twilio (pour WhatsApp — optionnel en développement)

Étapes d'installation

1. **Cloner et installer les dépendances**

```
git clone <repo>
cd pharmachat
npm install
```

2. **Configurer les variables d'environnement**

```
# Créer un fichier .env à la racine
DATABASE_URL="mongodb+srv://user:pass@cluster.mongodb.net/pharmachat"
TWILIO_ACCOUNT_SID="ACxxxxx"
TWILIO_AUTH_TOKEN="xxxxxx"
TWILIO_WHATSAPP_NUMBER="+14155238886"
```

3. **Synchroniser le schéma Prisma avec MongoDB**

```
npx prisma db push
```

4. **Démarrer le serveur de développement**

```
node server.js
```

L'application est accessible sur `http://localhost:3000`

5. **(Optionnel) Exposer pour Twilio**

```
ngrok http 3000
# Copier l'URL ngrok dans la console Twilio → webhook
```

Résumé du flux complet en une image

```
WhatsApp Patient
      │
      ▼
   Twilio (API WhatsApp Business)
      │ POST /api/webhook
      ▼
┌──────────────────────────────────┐
│         server.js                │
│   (HTTP + Socket.io sur :3000)   │
│                                  │
│  ┌──────────────────────────┐    │
│  │    Next.js App Router    │    │
│  │                          │    │
│  │  /api/webhook ──────────┼──► MongoDB (via Prisma)
│  │  /api/patients/[id]/msg ┼──► MongoDB + Twilio API
│  │                          │    │
│  │  Sidebar (Server Comp.) ┼──► MongoDB read
│  │  ChatContainer (Client) │    │
│  └──────────────────────────┘    │
│                                  │
│  globalThis.io (Socket.io) ─────┼──► React (new_message, unread_update)
└──────────────────────────────────┘
      │
      ▼
   Navigateur du Pharmacien
   (React UI mis à jour en temps réel)
```

Stack : Next.js 16 · TypeScript · Prisma · MongoDB · Socket.io · Twilio · Tailwind CSS v4

Avril 2026